Reproducing LiTS on a Mini Crossword

Individual Project 2 | FTEC5660 | Reproducibility Work | 2 November 2026

1 Target and scope

This project reproduces one archived result from LiTS, a framework for multi-step language-model search. The target is the archived demonstration labelled query 0, matched by board content to index 2 in the current bundled dataset. The repository reports 60% word accuracy and 0% exact-puzzle accuracy for this demonstration, with 133 model calls. I use the authors' search code and evaluation rather than implementing a substitute solver. The small modification reduces the MCTS iteration budget from 30 to 10.

Across three new trials, the 30-iteration condition achieved 86.7% mean word accuracy, compared with 66.7% for 10 iterations. These are measurements on one puzzle, not estimates of accuracy across the full dataset. The model and LiTS version differ from the archived run, so the original number is a contextual reference rather than a matched control.

Why this is an agentic system

The policy proposes a word placement, the environment updates the board, and a reward model evaluates the proposal. MCTS selects and expands paths, simulates continuations, and backpropagates rewards before choosing subsequent actions. Later calls depend on earlier actions and observations. This is a multi-step search process, not a single prompt-response test.

Source and provenance

Repository: https://github.com/xinzhel/lits-llm
Pinned upstream commit: 25e94d862cb4514c22d9a80a635e392461769df7
Original evidence: demos/demo_results/config.json and eval.log.
Data: demos/crosswords/data/mini0505.json, index 2 of 156 puzzles (the archived result calls it query 0).

The archived configuration identifies LiTS 0.3.1 and Claude 3.5 Sonnet (20240620). The checked-out package is 0.4.0. The protocol was written before the final trials; failed setup runs, the two-iteration pilot and an initial index-0 exploration are kept separate from the six matched-puzzle observations.

2 Setup and experimental design

Experiments ran on Windows using Python 3.14 in a dedicated virtual environment. Inference used the hosted DeepSeek API; no local GPU or model weights were required. Exact installed versions are saved in reproduction/environment-lock.txt. An API-only dependency list avoids unrelated local-model, geospatial and memory packages. This installation supports the selected task, not every LiTS feature. Policy temperature starts at 0.7 and increases by 0.2 on retries, capped at 1.0; policy top_p is 0.95. Reward sampling uses temperature 0.6 and the wrapper default top_p of 1.0. Policy output is capped at 1,024 tokens; binary reward outputs are capped at 10 tokens.

Setting	Value
Model and endpoint	deepseek-v4-flash; DeepSeek; https://api.deepseek.com
Generation	Thinking disabled; maximum 1,024 output tokens; upstream policy/reward sampling settings retained
Search	MCTS; 3 candidate actions; depth 10; rollout length 10; no memory or continuation
Controlled change	n_iters = 30 versus n_iters = 10
Repetition	Three trials per condition; local seeds 41, 42, 43
Data	Same puzzle and dataset hash in all six runs

The seeds control local Python and NumPy randomness. They are not sent as a deterministic sampling guarantee to the hosted provider. Runs are interleaved across conditions with up to three active processes. Consequently wall time includes network latency and possible service contention; token counts provide a more stable resource measure. Maximum output length is a cap, not the number of tokens requested on every call.

Prompts and the search algorithm are retained from the pinned checkout. Both conditions share the same compatibility adaptations: remove an unused torch import, tolerate missing IPython outside notebooks, use DeepSeek's thinking flag, omit a removed legacy search flag, and force UTF-8 file handling. These are not additional experimental treatment differences.

3 Metrics and evaluation

Word accuracy is the number of correct horizontal and vertical words divided by ten. Exact accuracy is one only when the complete board matches all ten answers. The original LiTS evaluation selects the output using its search returns, then applies the crossword goal checker. The study does not choose the best trial after seeing the answer key. Calls and tokens are summed from inferencelogger.log; elapsed time comes from run_manifest.json. Sample variance and standard deviation use n - 1 = 2 in the denominator.

4 Results

Condition	Seed	Words	Exact	Calls	Seconds
30 iterations	41	100.0%	100.0%	513	701.6
10 iterations	41	80.0%	0.0%	404	511.5
30 iterations	42	80.0%	0.0%	118	145.5
10 iterations	42	80.0%	0.0%	479	551.1
30 iterations	43	80.0%	0.0%	376	401.3
10 iterations	43	40.0%	0.0%	69	71.5

Measure	30 iterations	10 iterations
Mean word accuracy	86.7%	66.7%
Word accuracy SD	0.1155	0.2309
Word accuracy sample variance	0.01333	0.05333
Exact-puzzle rate	33.3%	0.0%
Mean API calls	335.7	317.3
Mean input tokens	117297	110758
Mean output tokens	13923	7757
Mean elapsed seconds	416.2	378.0

Relative to the archived 60% word score, the new baseline differs by +26.7 percentage points. Reducing the budget changes mean word accuracy by -20.0 percentage points and mean API calls by -18.3. These differences describe this experiment; three trials on a single puzzle do not establish a general benefit or harm.

The archived run used 55,630 input and 590 output tokens. Its displayed dollar estimate is not reused as a DeepSeek cost. This report makes no monetary claim without verified provider billing. Raw trial rows and sample statistics for every resource metric are supplied in trials.csv and summary.json.

5 What the results can and cannot establish

The repository workflow can be executed with a hosted alternative model, and the same task can be scored automatically. Matching the archived number would not prove that the original experiment had been recreated exactly: the model changed, the package evolved, and the archived record contains only one run. A different result likewise cannot be attributed to a single cause. The within-study comparison is stronger because both conditions share the model, dataset and code, with only the iteration limit changed.

A particularly important assumption appears in the environment reward. The policy and reward-model prompts do not include the answer key, but CrosswordsTransition.goal_check compares intermediate boards with the reference. EnvGroundedPRM then uses that progress in its reward. Search therefore benefits from privileged feedback. I retain it to reproduce the supplied setup, but the score should not be presented as performance on unseen crossword answers without such feedback.

Repeated or invalid actions remain in the logs and contribute to the measured cost. More iterations do not necessarily create more useful exploration. The model can revisit the same proposal or produce explanatory text when a short action is required. This helps explain why iteration budget, call count and accuracy need to be measured separately.

6 Debug diary

A data identity check found that archived query 0 matches current dataset row 2 by its full answer board. I preserved the earlier row-0 exploration and restarted all six final trials on the matching row. The protocol amendment records this correction; no trial was chosen because of a favourable score.

Installation first encountered AWS dependency backtracking. Removing the unused asynchronous provider from the API-only dependency list resolved that setup problem. Optional imports of torch and IPython also prevented lightweight execution and were adjusted without changing search behavior.

The archived terminate_on_terminal_node option was rejected by the newer version. It was omitted, while terminate_on_first_solution remained explicitly false. A further pilot produced empty final answers because thinking consumed the output budget; explicitly disabling the policy, evaluator and terminal-model thinking flags allowed usable actions. The corrected pilot scored 30% and is not counted as a final trial.

The first full batch exposed a Windows GBK encoding error when generated text contained a checkmark. Upstream swallowed the task exception and returned zero, and the evaluator could also return without a score file. The batch was archived and all six trials restarted under UTF-8. The runner now requires an evaluation artifact. Excluding these infrastructure failures was a documented batch-level decision, not selection based on accuracy.

7 Reproduction instructions and conclusion

From the repository root, create a Python virtual environment and install reproduction/environment-lock.txt, then run pip install -e . --no-deps. Put DEEPSEEK_API_KEY in an ignored .env file. The run scripts never print or commit the key. See README.md for exact commands and output locations.

Run reproduction/run_experiments.py for the six trials, reproduction/check_results.py for offline consistency checks, and reproduction/summarize.py to regenerate the numerical tables. Existing run directories are intentionally not overwritten: use a separate clone for a new batch. Each run retains its configuration, checkpoints, evaluation result, per-call usage and manifest. The summary includes SHA-256 hashes of its source files.

The main lesson is that reproducibility depends on the execution and evaluation contract as much as on the model. A successful exit code was insufficient evidence of a completed run. Small changes in model configuration could prevent actions from being produced at all. For future users, I recommend a short pilot, explicit encoding and provider settings, artifact checks, and a clearly defined comparison before spending resources on a larger benchmark.

A useful follow-up would repeat the budget comparison on a prespecified sample of additional puzzles. A separate experiment could remove answer-key-based reward shaping, but combining that change with the current budget change would make interpretation harder. Neither follow-up is claimed as completed here.

References and evidence

[1] LiTS source and README, pinned commit 25e94d862cb4514c22d9a80a635e392461769df7, https://github.com/xinzhel/lits-llm.
[2] Archived demonstration: demos/demo_results/config.json and eval.log in that commit.
[3] Crossword definitions and prompts: demos/lits_benchmark/crosswords.py; reward implementation: lits/components/reward/env_grounded.py.
[4] This project: reproduction/PROTOCOL.md, DEBUG_DIARY.md, runs/, analysis/trials.csv and analysis/summary.json.